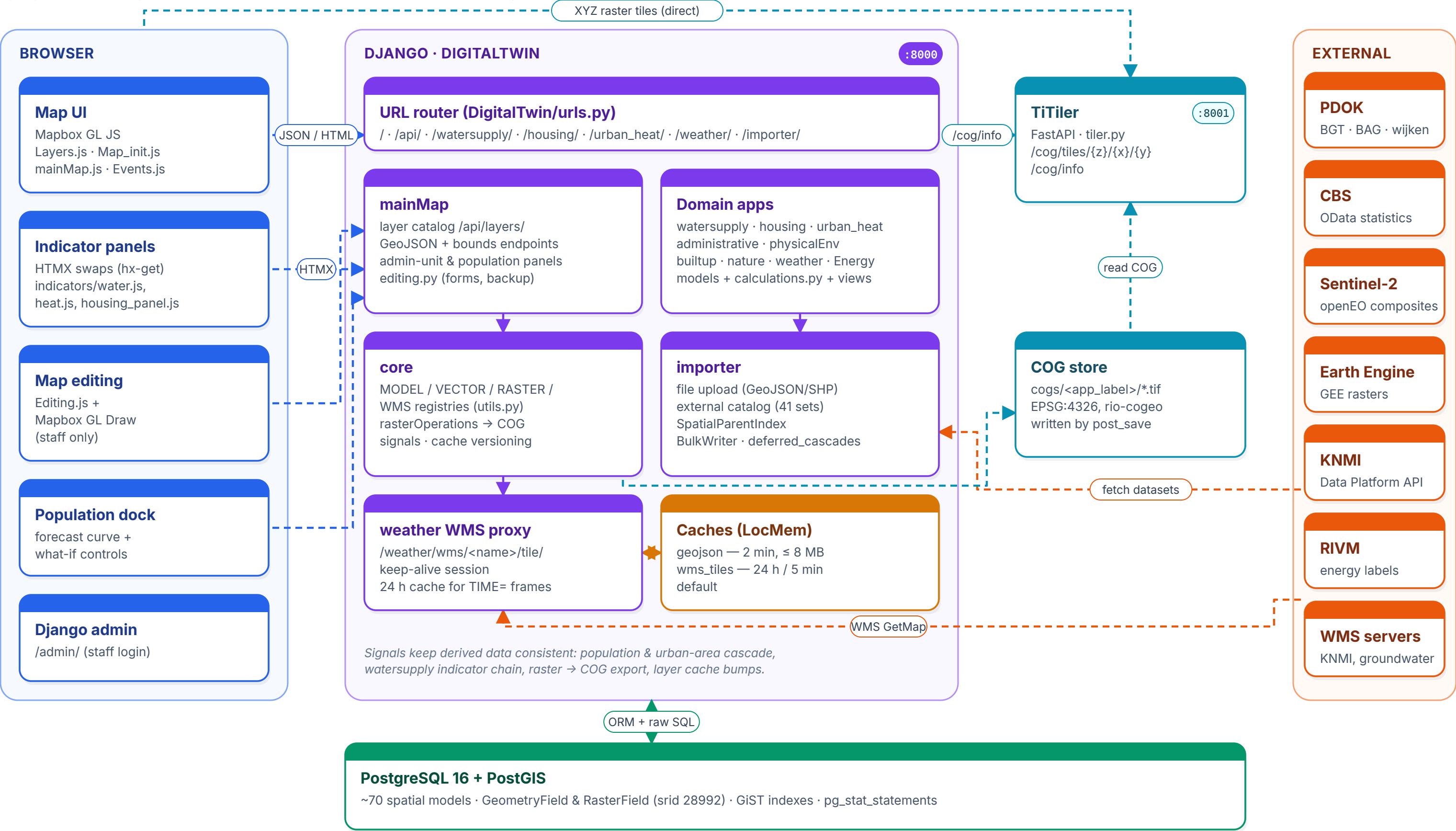


CrossTwin — System Overview

Django + PostGIS digital twin for Dutch urban data (EPSG:28992), with a separate TiTiler process for raster tiles



Django Apps & the Model Registry

core/utils.py::build_model_registry() scans nine domain apps; every generic endpoint is driven by the registries

Django Vector Raster WMS

DOMAIN APPS (ALLOWED_APPS) — MODELS · CALCULATIONS.PY · VIEWS

administrative 5
Province > City >
District > Neighborhood
PopulationProjection

physicalEnv 12
LandCover vec + raster
DEM · DSM · imagery
materials, env. costs

watersupply 17
extraction · treatment
pipes · NRW · OPEX
SupplySecurity

housing 8
supply/demand · HPI
mortgage · rentals
affordability

urban_heat 11
UTCI · PET · Tmrt
LST · SVF · SUHII
NBS · WBGT

builtup 6
ZoningArea · Street
Park · Facility
Building · Property

nature 6
green / blue
layers

weather 4
WMS layers
time steps

Energy 2
energy labels

Number tags = model classes per app. Indicator dashboards (watersupply, housing, urban_heat) follow calculations.py → views._build_indicators() → HTMX partial; MOCK_DATA fills in when a province is missing.

DAG edges documented in calculations.py docstrings → core/DAG.dot (DPSIR)

scan models

build_model_registry()
core/utils.py — scans allowed_apps at startup

has GeometryField, no RasterField → VECTOR
has RasterField → RASTER · 'WMS' in key → WMS

MODEL_REGISTRY

all models

VECTOR_REGISTRY

GeoJSON, bounds, editing

RASTER_REGISTRY

COG export, tile URLs

WMS_REGISTRY

WMS layer catalog

importer

field mapping, upsert keys

mainMap

/api/layers/, GeoJSON, editing

core.signals

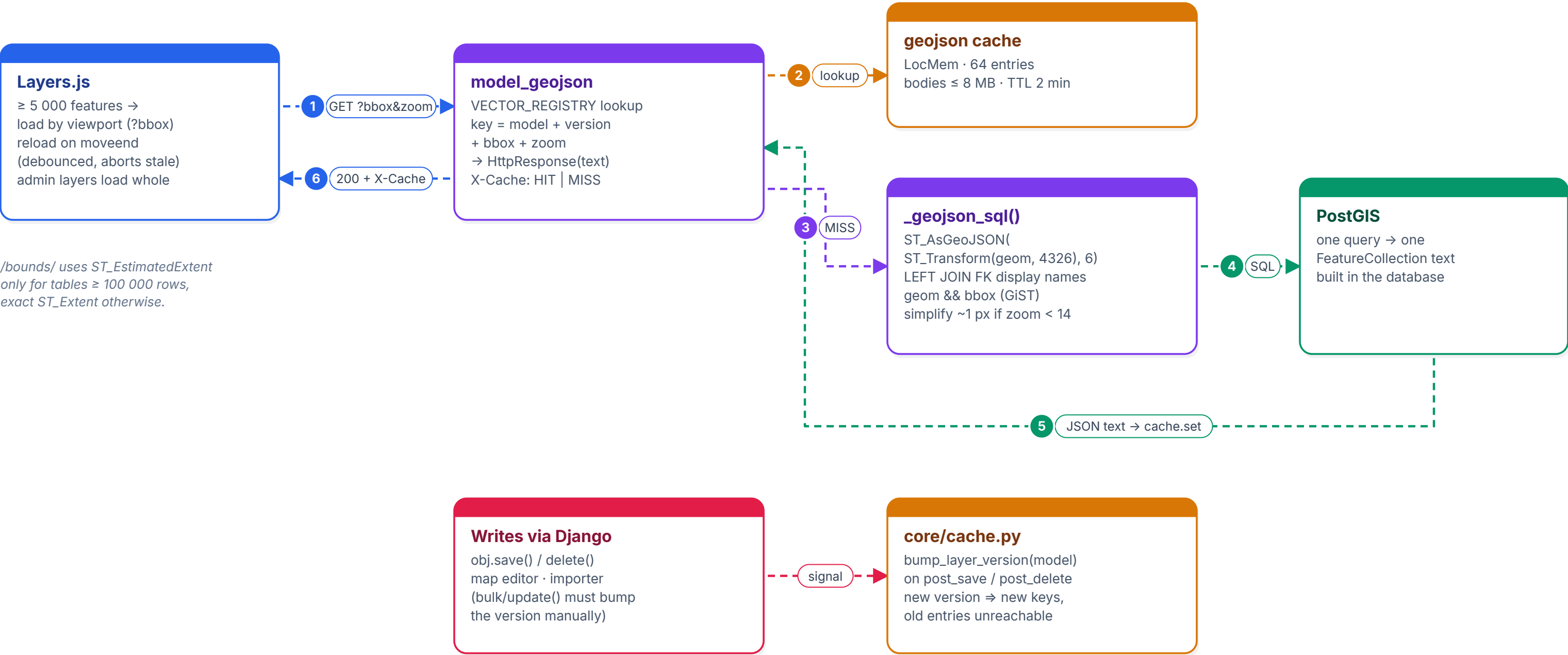
post_save → COG, cache bump

weather

WMS catalog, tile proxy

Vector Layer Request — GeoJSON Path

mainMap.views.model_geojson: one SQL statement per request, cached by per-model version

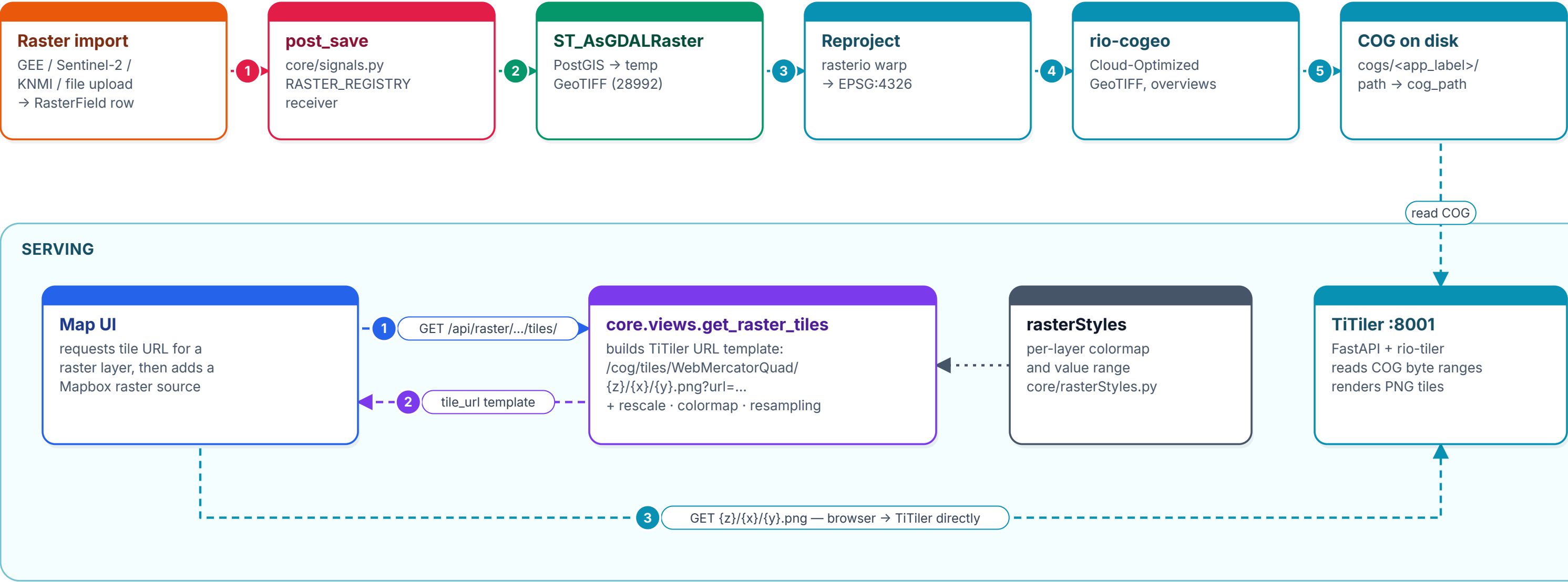


/bounds/ uses ST_EstimatedExtent
only for tables ≥ 100 000 rows,
exact ST_Extent otherwise.

Edits from QGIS/psql bypass Django signals → visible only after the 2-min TTL expires.
LocMem is per process: with several workers, point CACHES at Redis so a bump reaches all of them.

Raster Pipeline — PostGIS Raster → COG → Tiles

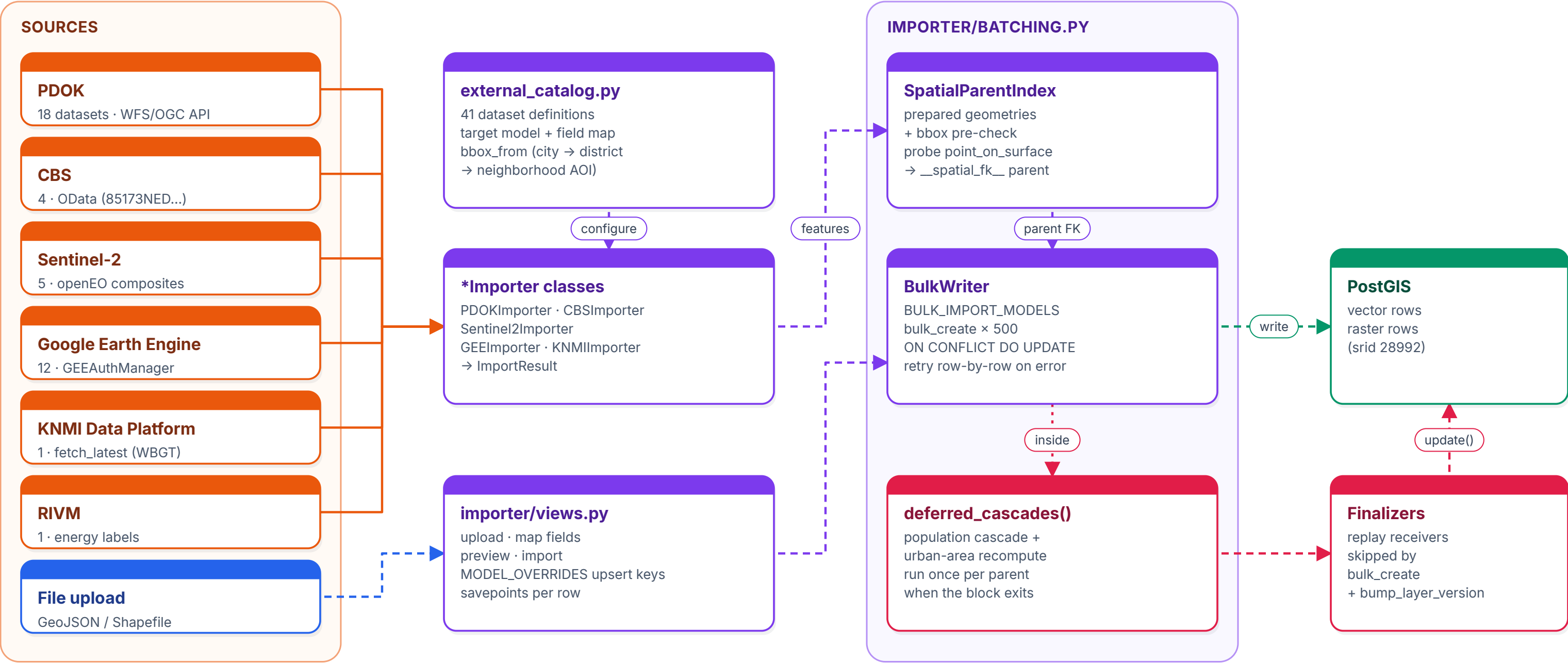
core/rasterOperations.py::export_raster_to_cog() runs from a post_save signal on every RASTER_REGISTRY model



Heavy libs (rasterio, rio-cogeo, numpy) are imported inside export_raster_to_cog(), never at module level, so Django startup and the URLconf stay fast; tests patch core.rasterOperations.export_raster_to_cog.

Data Import Pipeline

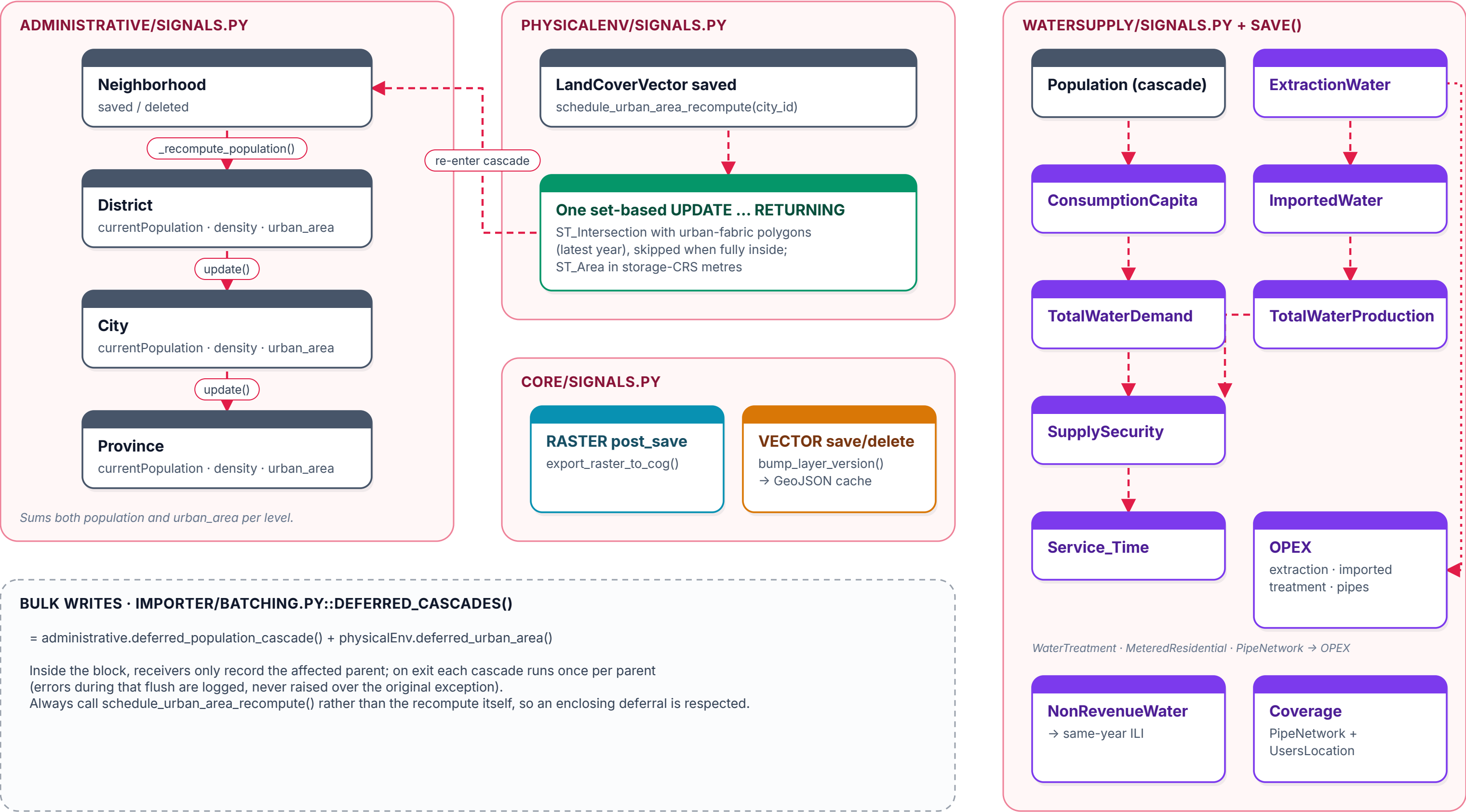
Two entry points — file upload and the catalog-driven external importers — share batching and deferred cascades



A model may be in BULK_IMPORT_MODELS only if it has no save() override (BulkImportRegistryTests);
adding a post_save receiver to one means adding it to its finalizer too.
Known gap: _generic_import (file upload) maps geometry/raster fields only — attribute columns are not imported.

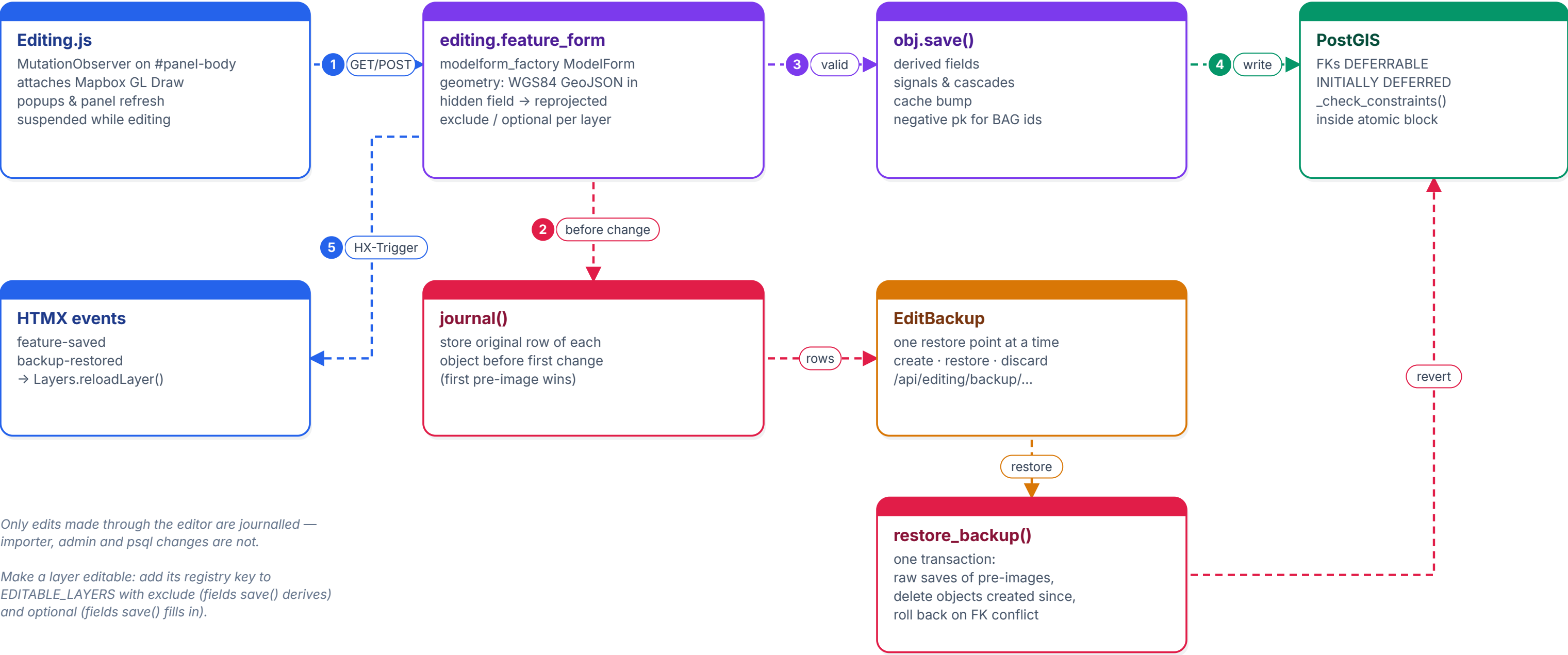
Signal-Driven Derived Data

post_save / post_delete receivers keep aggregates and indicator chains consistent; update() avoids recursion



Map Editing & What-If Backup

Staff edit EDITABLE_LAYERS on the map; every change goes through obj.save() and is journalled for one-click restore



Only edits made through the editor are journalled —
importer, admin and psql changes are not.

Make a layer editable: add its registry key to
EDITABLE_LAYERS with exclude (fields save() derives)
and optional (fields save() fills in).